Mini RAG

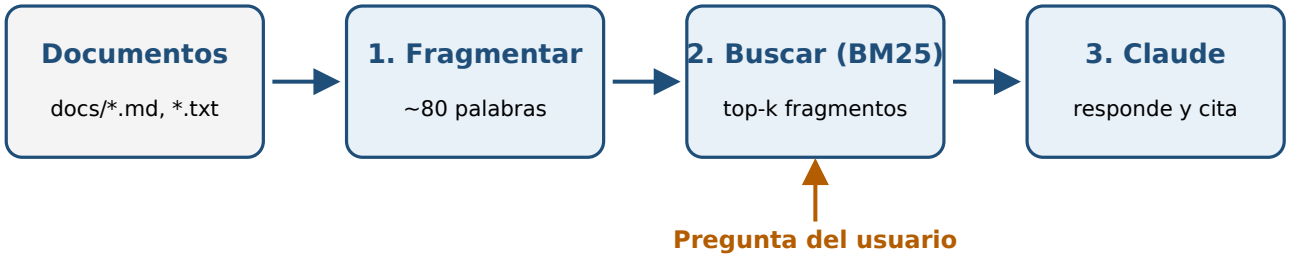
Cómo funciona, paso a paso

¿Qué es un RAG?

RAG significa Retrieval-Augmented Generation (generación aumentada por recuperación). Un modelo de lenguaje como Claude sabe mucho, pero no conoce **tus** documentos. Un RAG lo resuelve en dos fases: primero **busca** los trozos de tus documentos que tienen que ver con la pregunta, y después se los **entrega** al modelo para que responda basándose en ellos.

> Es como un examen a libro abierto: en vez de responder de memoria, el estudiante busca la página correcta del libro y responde a partir de ella. Así las respuestas son más precisas, actualizadas y se puede comprobar de dónde salió cada dato.

Vista general



Todo el sistema vive en un solo archivo, `rag.py`, y tiene tres pasos. Los pasos 1 y 2 son Python puro y no necesitan internet; solo el paso 3 llama a la API de Claude.

Paso 1 · Cargar y fragmentar

La función `cargar_fragmentos()` lee todos los archivos `.md` y `.txt` de la carpeta `docs/`. Cada archivo se separa en párrafos (por líneas en blanco) y los párrafos se van juntando hasta llegar a unas 80 palabras. Cada grupo es un **fragmento**, que guarda su texto y el nombre del archivo de donde salió.

¿Por qué fragmentar en vez de usar el documento entero?

- Se busca con más precisión: un fragmento pequeño trata de un solo tema.
- Se le envía a Claude solo lo relevante, lo que es más barato y rápido.
- Se puede citar exactamente de qué archivo salió cada dato.

Paso 2 · Buscar con BM25

Para decidir qué fragmentos son relevantes se usa **BM25**, el algoritmo clásico de los buscadores. Antes de comparar, el texto se **normaliza**: todo en minúsculas, sin tildes y sin palabras vacías ("el", "de", "que"...). Así "Júpiter" y "jupiter" cuentan como la misma

palabra.

Después, cada fragmento recibe una puntuación según tres ideas:

Idea	Qué significa	Ejemplo
Frecuencia (TF)	Si una palabra de la pregunta aparece varias veces en el fragmento, suma más puntos (pero con un tope: repetirla 20 veces no vale 20 veces más).	Un fragmento que dice "café" 4 veces puntúa más que uno que lo dice 1 vez.
Rareza (IDF)	Las palabras que aparecen en pocos fragmentos valen más que las muy comunes.	"espresso" solo sale en un fragmento: es una pista muy fuerte.
Longitud	Se penalizan un poco los fragmentos largos, que tienen más palabras "por casualidad".	Un fragmento de 30 palabras con el término gana a uno de 200.

La fórmula, para quien quiera verla (k_1=1.5 y b=0.75):

```
puntaje = Σ  IDF(t) · tf · (k1 + 1) / ( tf + k1 · (1 − b + b · largo / largo_medio) )
IDF(t)  = log( 1 + (N − df + 0.5) / (df + 0.5) )
```

Se devuelven los **k** fragmentos con mayor puntuación (3 por defecto).

Paso 3 · Generar la respuesta con Claude

La función `responder()` envía a Claude los fragmentos encontrados junto con la pregunta. Cada fragmento va etiquetado con su fuente:

```
<contexto>
<fragmento id="1" fuente="sistema_solar.md">
Júpiter es el planeta más grande del sistema solar...
</fragmento>
</contexto>

Pregunta: ¿Cuál es el planeta más grande?
```

Las instrucciones (system prompt) le piden a Claude tres cosas:

- Responder en español usando **solo** la información de los fragmentos.
- Citar la fuente entre corchetes, por ejemplo `[sistema_solar.md]`.
- Decir claramente cuando los fragmentos no contienen la respuesta, en vez de inventarla.

Se usa el modelo `claude-opus-5-5` con esfuerzo bajo (`effort: low`), porque responder a partir de un contexto dado es una tarea sencilla. Si Claude declinara responder, la API reintenta automáticamente con un modelo de respaldo (`fallbacks="default"`).

Cómo usarlo

```
pip install -r requirements.txt
export ANTHROPIC_API_KEY=sk-ant-...

python rag.py "¿Cuál es el planeta más grande?"   # una pregunta
python rag.py                                     # modo interactivo
python rag.py --solo-buscar "espresso"            # solo búsqueda, sin API
python rag.py -k 5 --docs mis_docs/               # otra carpeta, más fragmentos
```

Ejemplo real de la fase de búsqueda:

```
$ python rag.py --solo-buscar "¿Cuál es el planeta más grande?"
Indexados 5 fragmentos de docs

  [2.12] sistema_solar.md: # El sistema solar  El sistema solar está formado...
  [1.50] sistema_solar.md: Marte es conocido como el planeta rojo...
```

El primer fragmento, que contiene "Júpiter es el planeta más grande", obtiene la mayor puntuación. Para usar tus propios documentos, basta con copiarlos a `docs/`.

Limitaciones e ideas para mejorarlo

Limitación	Mejora posible
BM25 busca palabras exactas: "auto" no encuentra "coche".	Usar **embeddings** (vectores de significado) y búsqueda vectorial.
El índice se recalcula en cada ejecución.	Guardarlo en disco o en una base de datos vectorial.
Solo lee .md y .txt.	Añadir lectores de PDF, Word o páginas web.
Cada método falla en casos distintos.	Búsqueda **híbrida**: combinar BM25 y embeddings.